

Machine Vision Final Project Report

J.J. Moncus

2026-01-02

Initial Idea: Pose Estimation, and Why I Didn't Pursue It

My first two ideas both involved some degree of pose estimation. In the end, the first was implausible given the data we're working with, while the second fell outside the bounds of what was allowed in the assignment.

First: "Follow the Elbow"

Algorithm

- 1) For every frame:
 - Use template images to identify the maximum-likelihood estimate (x,y) coordinates of the right shoulder, elbow, and wrist
 - Fit a line connecting the shoulder to the elbow, and fit another line connecting the elbow to the wrist.
 - Calculate the angle between the two lines.
 - Each frame is thus reduced to a one-dimensional scalar (i.e. the angle of the elbow), and we no longer have to hold onto the pixel information.
- 2) Count the number of instances the elbow angle exits and re-enters "threshold zones" near 0° (for "closed") and 180° (for open).
 - Since arms naturally do not close all the way, the precise angle threshold for closed would have to be picked by inspecting the training data, perhaps somewhere near 10° to start. The same goes for the open position - some individual's may be unable to straighten their arms all the way, so we might declare open to be anywhere above 170° .

Why I think this wouldn't work

- 1) The shoulder, elbow, and wrist all change orientation as they move, whereas the template image used to track them would likely be fixed (probably based on what each joint typically looks like in the initial push-up top position). Thus, we might lose track of the joint coordinates as the

person executes the pushup, which would make computing lines through those points very noisy.

- 2) Not all videos even feature the right arm. We might have to start with some first layer, identifying what size of the person is visible, then apply a different template for each side, which seemed needlessly complicated.
- 3) Some videos are shot such that the elbow angle isn't obvious (like from the front, or from above). In these cases, fitting useful lines could be very difficult.
- 4) This method would be sensitive to camera movement. If the elbow angle in plane-space looks like it's changing just because the camera rotates around the person, the model might see pushups where there aren't any.

In short, this approach *might* make sense if all videos were shot from the side, and if the orientation of the joints don't change much as they move (i.e. if it's still easy to recognize that a shoulder is a shoulder, whether the arm is straight or bent). Given none of this is the case, I scrapped this plan pretty quickly.

Second: "Follow the Chest"

Algorithm

- 1) For every frame:
 - Use a pre-trained pose estimation model to calculate the (x, y) coordinates of the two shoulders and the two hips.
 - Take the average of these to get the (x, y) coordinates of the "torso centroid"
 - Each frame is thus reduced to a two-dimensional scalar (i.e.), and we no longer have to hold onto the pixel information.
- 2) Using the trunk centroid for the first frame, declare some (x, y) range where we decide the person is in the "top" position.
 - We would have to decide this by inspection, but start with something conservative, like 20 pixels in each direction.
- 3) Across frames, count the number of times the trunk centroid leaves the top range and comes back again.

Why I think this wouldn't work

- 1) This method would be highly sensitive to any camera movement. If the trunk centroid moves in the frame simply because the camera rotates around the person, the model may see pushups where there aren't any. And because we're not passing any other pixel-information, the model wouldn't be able to which trunk movements correspond to camera movements.
- 2) While it is helpful to know that the test data will only contain whole integer pushups, this method could be sensitive to small torso movements that don't count as whole pushups (like, say, if the person hunches or stretches their back without moving their arms).

This method *might* work if all videos are shot with stable cameras and if the subjects don't wiggle much in between pushups. But we can't be confident that that will be the case in the test data.

Summary

All told, pose estimation seems intuitively helpful but insufficient. To be successful, my hunch was that the model needs to keep track of both the person AND how the person relates to other elements in the frame, like the ground or the sky. For that, we need to pass full pixel-information into our first layer.

Learning about the data

To get a sense of our training data, I watched most of our training videos and inspected some meta-data:

- All videos seem to be shot with 24 frames-per-second, for a total length of roughly 90 to 360 frames (i.e. roughly 3.5 to 10 seconds).
- Most videos are shot in wide-screen, meaning if we squish videos to a 224x224 square resolution for training, the subject will look squished. Hopefully the push-up movement would still be apparent.
- Videos are shot from all angles (from the side, from the front, from above), and sometimes with camera movement, which validates that just using pose estimation for the subject would have been tricky.
- Since pushups are relatively smooth movements, our model may be able to stride over frames (i.e. for each video, only look at every 5th or 10th frame) and still see the subject's movement. This way, we could save a lot of memory overhead.

First Real Idea: A Basic CNN for both space and time

To track the subject's pushup movement over space and time, I proposed a two-part architecture

- A series of 2D-convolutions on each frame, to extract spatial features about the subject and its surroundings.
- A series of 1D-convolutions across frames, to see how those spatial features changed over time.

The open question was whether the spatial, 2D-Conv layers would learn enough about what a person is, and what it means for that person to move, with no pre-training.

Disclaimer

I reached for a CNN before even trying a fully-connected MLP.

An MLP has much weaker inductive bias than a CNN, since the MLP does not presume translational equivariance over the input data. This is helpful sometimes in that an MLP could potentially learn far more complicated relationships between inputs and outputs, but at the cost of much larger memory overhead. However, the CNN would have the advantage of learning to understand pushups regardless of where the subject lives in the frame, whereas an MLP would

have to relearn this for every new subject base location. The weak inductive bias also makes the MLP more likely to overfit. For this reason, along with our memory footprint limits, a CNN seemed preferable.

In hindsight, starting with an MLP would have been helpful to get a baseline understanding for how difficult the our learning problem is. If even a relatively-deep MLP struggles to acheive high training accuracy, then our CNN would likely be insufficient. Luckily, this turned out to not be necessarily.

The Result: Vanishing Gradients

Every attempt to use a CNN for building spatial features resulted in the model only ever predicting the mean label for each batch.

Below is the summary of the first attempt, which I then modified a number of ways in different combinations:

Original

Loss: Mean Squared Error, for regression

Stage	Input Shape	Operations	Output Shape
Spatial Encoder	$(B \times T, 3, 224, 224)$	6x "Conv2d (Stride 2) + ReLU" + Linear(640, 2)	$(B \times T, 2)$
Temporal Conv	$(B, 2, T)$	3x Conv1d (Stride 2) + ReLU	$(B, 18, 13)$
Classifier	$(B, 234)$	Flatten + Linear	$(B, 1)$

Modifications

- added Batch Normalization to each convolutional block in both spatial and temporal stages.
- several tweaks to the Linear bottlenecks in each stage to be less aggressive (and adjusting the convolutions' inputs and outputs to accomodate the shape changes).
- Increased temporal convolutions kernel size from 3 to 5, to try to capture more movement in each step.

Across 5 attempts with different adjustments, the model only ever predicted the mean of the test batch.

Second Real Idea: ResNet for Spatial Features

Replacing the CNN in the spatial stage with a pre-built ResNet finally got the model training.

Successful training! 84% accuracy on full training set, but only 50% on test data

overfitting , lets try to fix Unfroze final layer of resnet Also added weight decay to optimizer Achieved full eval on training set of 86%, but only 43% accuracy on test set (even worse) Lets try again but with larger weight decay, 1e-3 86% train acc, 44% val acc, not good Lets try refreezing the weights from the

Stage	Input Shape	Operations	Output Shape
Spatial Backbone	$(B \times T, 3, 224, 224)$	Pre-trained ResNet18 (Frozen)	$(B \times T, 512, 1, 1)$
Temporal Processor	$(B, 512, T)$	3x [Conv1d + BatchNorm + LeakyReLU]	$(B, 32, 13)$
Classifier	$(B, 416)$	Flatten + Linear	$(B, 1)$

Table 1: Architecture Summary using Transfer Learning Backbone

resnet backbone, and back to small weight decay, 1e-4. Let's also try fewer epochs, 12 86% train, 52% test, not great